



02. C++ 프로그래밍의 기본

Heesung Oh

<https://github.com/3dapi>



- 이름 공간과 이름 충돌 방지
- 출력 스트림과 서식 지정
- 입력 스트림과 상태 복구
- 타입의 크기, 정렬과 표현 범위 확인
- 문자 인코딩과 부동소수점 오차
- 초기화, const, 컴파일 타임 기능
- 타입 표현, 함수 확장, 참조

C의 절차적 문법



C++의 안전한 표현과
표준 라이브러리



클래스와 객체지향
프로그래밍으로 확장





2.1 이름 공간 (Scope)

- 이름 공간은 함수, 타입과 변수의 이름을 별도의 범위로 묶음
- 서로 다른 라이브러리나 모듈의 이름 충돌 방지
- 범위 지정 연산자 ::로 이름 공간 안의 이름 사용

```
namespace Audio
{
    void Initialize()
    {
    }
}

namespace Graphics
{
    void Initialize()
    {
    }
}

Audio::Initialize();
Graphics::Initialize();
```

표현	의미
Audio::Initialize()	Audio 이름 공간의 함수
Game::Player	Game 이름 공간의 타입
::value	전역 범위의 이름



2.1.3 std 이름 공간과 using 선언

- std C++ 표준 라이브러리 이름 공간
- std::는 범위를 분명하게 표현
- using 선언으로 개별적 사용
- using namespace std;
 - ◆ 헤더 파일에 작성 적용할 때, 영향 고려

```
#include <iostream>
int main()
{
    using std::cin;
    using std::cout;

    int value = 0;

    cin >> value;
    cout << value << '\n';
}
```

사용	특징
std::cout	출처가 분명함
using std::cout;	특정 이름만 가져옴
using namespace std;	이름 검색 범위가 넓어짐



- `std::cout`은 표준 출력 스트림
- 출력 연산자 `<<`로 여러 타입의 값 출력
- 값의 타입에 맞는 출력 동작 선택
- `printf()`처럼 형식 지정자를 직접 맞추지 않음

```
#include <iostream>
int main()
{
    int level = 10;
    double speed = 3.5;

    std::cout << level << '\n';
    std::cout << speed << '\n';

    std::cout << "Level: " << level
               << ", Speed: " << speed << '\n';
}
```

표현	동작
<code>'\n'</code>	줄 바꿈
<code>std::endl</code>	줄 바꿈 + 버퍼 비움
<code>std::flush</code>	출력 버퍼 비움



● 기본 조정자(Manipulator)

조정자	의미
endl	줄 바꿈 + 버퍼 비움
dec, hex, oct	10진수, 16진수 8진수
nboolalpha, boolalpha	false true 출력
nshowbase, showbase	진법 접두사 출력
nshowpos, showpos	양수 앞에 + 출력
nuppercase, uppercase	16진수 문자와 지수 표기 대문자
left, right, internal	정렬
fixed, scientific	고정 소수점, 지수 형식
defaultfloat, hexfloat	출력 형식 기본, 16진수 지수 표기
nshowpoint, showpoint	소수점과 뒤쪽 0 표시





● 기본 조정자(Manipulator)

```
int value = 255;

std::cout << std::dec << value << '\n';
std::cout << std::hex << value << '\n';
std::cout << std::oct << value << '\n';

std::cout << std::showbase;
std::cout << std::hex << 255 << '\n'; // 0xff
std::cout << std::oct << 255 << '\n'; // 0377
```

```
bool result = true;

std::cout << result << '\n';
std::cout << std::boolalpha << result << '\n';
std::cout << std::showpos
          << 10 << '\n';
```



- `#include <iomanip>`

조정자	의미
<code>std::setw()</code>	이어지는 값 하나의 최소 필드 폭
<code>std::setfill()</code>	남는 공간의 문자
<code>std::setprecision()</code>	출력 자릿수 지정
<code>std::setbase(8, 10, 16)</code>	출력 진법 변경
<code>std::setfill()</code>	남는 공간의 문자

```
#include <iomanip>
std::cout << std::setw(6) << 10 << '\n';
std::cout << std::setfill('0')
          << std::setw(5)
          << 42 << '\n';
```

```
double value = 1234.56789;
std::cout << std::fixed << value << '\n';
std::cout << std::scientific << value << '\n';
std::cout << std::hexfloat << value << '\n';
std::cout << std::defaultfloat;
```





- `std::cin`은 표준 입력 스트림
- 입력 연산자 `>>`로 값을 읽음
- 변수의 타입에 맞는 입력 동작 선택
- 일반 변수의 주소를 전달하지 않음
- 입력 데이터가 변수 타입과 맞지 않으면 실패 상태 설정

C	C++
<code>scanf("%d", &level);</code>	<code>std::cin >> level;</code>
형식 지정자 필요	타입에 따라 입력 동작 선택
주소 전달 필요	일반 변수 그대로 사용

```
int level = 0;
double speed = 0.0;
std::cin >> level >> speed;
```





2.3.2 std::string

- C++의 일반 문자열 표현은 std::string
- 문자열의 길이와 저장 공간을 객체가 관리
- 대입, 복사, 비교, 결합을 일반 변수와 비슷하게 작성
- strcpy(), strcat() 대신

```
#include <iostream>
#include <string>
using std::cout;
using std::string;

int main()
{
    string firstName = "Dark";
    string lastName = "Knight";
    string name = firstName + " " + lastName;
    string message = "Player";
    message += ": ";
    message.append(name);
    message.append(3, '!');
    cout << message << '\n';
}
```



2.3.2 cin >>와 getline()

```
std::string name;  
std::cin >> name;
```

- ◆ 공백 전까지 하나의 단어 입력
- ◆ Dark Knight를 입력하면 Dark만 저장

```
std::string name;  
std::getline(std::cin, name);
```

- ◆ 줄 바꿈을 만날 때까지 한 줄 입력
- ◆ 공백을 포함한 문자열 저장

```
int level = 0;  
std::string name;  
std::cin >> level;  
std::getline(std::cin >> std::ws, name);
```

- ◆ std::ws는 입력 앞쪽의 공백과 줄 바꿈을 제거
- ◆ 앞쪽 공백을 보존해야 하면 ignore() 사용



- 입력 데이터가 변수 타입에 맞지 않으면 실패 상태 설정
- 입력 연산 자체를 조건식으로 검사 가능
- `while (!std::cin.eof())`보다 입력 연산을 반복 조건으로 사용

함수	의미
<code>good()</code>	오류 상태가 없음
<code>eof()</code>	입력의 끝에 도달
<code>fail()</code>	형식 오류 또는 입력 실패
<code>bad()</code>	복구하기 어려운 스트림 오류

```
int value = 0;
if (std::cin >> value)
{
    std::cout << "입력한 값: " << value << '\n';
}
else
{
    std::cout << "정수를 입력하지 않았습니다.\n";
}
```



2.3.6 clear()와 ignore()

- 실패 상태가 설정되면 이후 입력도 정상 처리되지 않음
- clear()는 스트림 상태 플래그를 정상 상태로 되돌림
- ignore()는 입력 버퍼에 남은 문자를 버림
- 두 함수의 역할을 구분해야 함

```
#include <iostream>
#include <limits>
int main()
{
    int value = 0;
    while (true)
    {
        std::cout << "정수를 입력하세요: ";
        if (std::cin >> value)
        {
            break;
        }
        std::cout << "잘못된 입력입니다.\n";
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
    std::cout << "입력한 값: " << value << '\n';
}
```



2.4 타입의 크기와 범위 확인

- C/C++ 표준은 기본형 타입에 대해 바이트 크기 정하지 않음
- 표준은 최소 범위와 타입 사이의 크기 관계 규정
- 실제 크기와 정렬 방식은 CPU, OS, 컴파일러와 ABI에 따라 달라짐

데이터 모델	대표 환경	특징
LLP64	64비트 Windows	int, long 32비트, 포인터 64비트
LP64	64비트 Linux, macOS	int 32비트, long, 포인터 64비트

```
std::cout << sizeof(int) << '\n';  
std::cout << sizeof(double) << '\n';  
  
int value = 100;  
std::cout << sizeof(value) << '\n';
```



- sizeof(char)는 항상 1
- 1바이트를 구성하는 비트 수는 CHAR_BIT로 확인
- sizeof의 피연산자는 일반적으로 실행되지 않음

```
#include <climits>
#include <iostream>

int main()
{
    int value = 10;
    std::size_t size = sizeof(++value);
    std::cout << value << '\n'; // 10
    std::cout << sizeof(char) << '\n';
    std::cout << CHAR_BIT << '\n';
}
```

- 배열에 sizeof를 적용하면 배열 전체의 바이트 수
- 함수 매개변수의 배열 표기는 포인터로 조정됨



정수형

```
#include <limits>
std::cout << std::numeric_limits<int>::min() << '\n';
std::cout << std::numeric_limits<int>::max() << '\n';
std::cout << std::numeric_limits<unsigned int>::max() << '\n';
```

부동소수점형

```
std::cout << std::numeric_limits<double>::min() << '\n';
std::cout << std::numeric_limits<double>::lowest() << '\n';
std::cout << std::numeric_limits<double>::max() << '\n';
```

표현	의미
min()	0보다 큰 가장 작은 정규화 값
lowest()	가장 작은 유한값
max()	가장 큰 유한값
digits10	십진 유효 자릿수 기준
max_digits10	문자열 출력 후 재입력 복원 기준



- 문자형의 크기, 문자 인코딩은 다른 개념
- 문자형은 인코딩을 구성하는 코드 단위 저장
- 문자 인코딩은 문자를 하나 이상의 코드 단위로 변환하는 규칙

표기	타입	용도
'A'	<code>char</code>	일반 문자 코드 단위
<code>L'A'</code>	<code>wchar_t</code>	와이드 문자 코드 단위
<code>u8'A'</code>	<code>char8_t</code>	UTF-8 코드 단위
<code>u'A'</code>	<code>char16_t</code>	UTF-16 코드 단위
<code>U'A'</code>	<code>char32_t</code>	UTF-32 코드 단위

```
char character = 'A';  
char8_t utf8Character = u8'A';
```





2.6 부동소수점형과 오차

- 부동소수점형은 한정된 비트 안에서 넓은 범위의 실수 표현
- 대부분의 값을 정확한 값이 아니라 근사값으로 저장
- 2진수로 유한하게 표현할 수 없는 십진 소수는 가까운 표현 가능 값으로 저장
- 구현 정의만 있고, IEEE754 준수를 강제하지 않음

```
#include <iomanip>
#include <iostream>
double first = 0.1;
double second = 0.2;
double result = first + second;
std::cout << std::setprecision(17) << result << '\n';

// 출력:
// 0.300000000000000004
```

- 연산이 잘못된 것이 아니라 한정된 비트로 실수를 표현한 결과



2.6.2 == 비교의 의미

```
double first = 0.5;  
double second = first;
```

```
bool result = first == second;
```

- 같은 값을 복사했거나 2진수로 정확히 표현되는 값은 정확한 비교가 목적에 맞을 수 있음

```
double value = 0.1 + 0.2;  
bool result = value == 0.3;
```

- 여러 연산을 거친 결과를 특정 십진수와 비교할 때는 오차 고려 필요
- 수학적으로 같은 값이어도 **false** 가능

비교 기준	사용 상황
절대 오차	값의 범위가 일정하거나 0에 가까운 값
상대 오차	값의 크기가 크게 달라지는 값



2.6.3 절대 오차와 상대 오차

```
#include <algorithm>
#include <cmath>

bool NearlyEqual(double first, double second, double absoluteTolerance
                , double relativeTolerance)
{
    if (first == second)
    {
        return true;
    }
    if (!std::isfinite(first) || !std::isfinite(second))
    {
        return false;
    }

    double difference = std::abs(first - second);
    if (difference <= absoluteTolerance)
    {
        return true;
    }

    double scale = std::max(std::abs(first), std::abs(second));
    return difference <= scale * relativeTolerance;
}
```

- 허용 오차는 계산 목적과 데이터 범위에 맞게 결정



- 초기화는 객체가 만들어질 때 처음 값을 정하는 과정
- 대입은 이미 존재하는 객체의 값을 변경하는 연산
- 코드 모양이 비슷해도 초기화와 대입은 다른 과정

```
int value = 10; // 초기화  
value = 20;    // 대입
```

● C++ 초기화 문법

```
int first = 10;    // 대입 형식 초기화  
int second(10);    // 직접 초기화  
int third{ 10 };   // 중괄호 초기화(List-initialization)
```

- 기본형에서는 모두 10으로 초기화
- 클래스 타입에서는 선택되는 생성자가 달라질 수 있음



- 기본형, 배열, 구조체와 클래스 객체에 일관된 형태로 사용
- 값이 손실될 수 있는 일부 축소 변환을 허용 불가

```
int count{ 10 };  
double speed{ 3.5 };  
bool enabled{ true };  
  
// int value{ 3.5 }; // 오류  
int value = 3.5;
```

- 대입 형식 초기화에서는 변환이 허용되며 소수점 이하



- const는 객체를 통해 값을 변경하지 않겠다는 뜻을 타입에 포함
- 선언할 때 초기화 필요
- 컴파일 시점 상수라는 뜻은 아님
- 초기값은 컴파일 시점 또는 실행 시점에 결정 가능

```
const int maxCount{ 100 };  
// maxCount = 200; // 오류  
  
int values[maxCount]; // 가능  
  
const int runtimeCount{ GetCount() };  
// int otherValues[runtimeCount]; // 오류
```

객체	값 결정 시점	변경
const int maxCount{100}	컴파일 시점 가능	불가능
const int runtimeCount{GetCount()}	실행 시점	불가능



- 컴파일 시점에 사용할 상수임을 명확하게 표현
- constexpr 변수는 암시적으로 const
- 초기값을 컴파일 시점에 계산할 수 있어야 함

```
constexpr int maxCount{ 100 };  
int values[maxCount];  
  
constexpr int Square(int value)  
{  
    return value * value;  
}  
  
constexpr int size{ Square(10) };  
static_assert(size == 100);
```

- 상수 표현식이 필요한 위치에서는 컴파일 시점 계산
- 실행 시점 값을 전달하면 일반 함수처럼 실행 시점 계산 가능
- const는 변경 제한, constexpr는 컴파일 시점 사용 가능성 표현



```
int first{ 10 };  
int second{ 20 };  
  
const int* pointerToConst{ &first };  
int* const constPointer{ &first };  
const int* const constPointerToConst{ &first };
```

선언	가리키는 값 변경	다른 주소 대입
const int* pointerToConst	불가능	가능
int* const constPointer	가능	불가능
const int* const constPointerToConst	불가능	불가능

읽는 방법

- const int* : const int를 가리키는 포인터
- int* const : int를 가리키는 const 포인터
- const int* const : const int를 가리키는 const 포인터



- C++는 C의 저수준 제어 기능을 유지
- 변환의 의도와 타입의 의미를 코드에 명확하게 표현
- C 스타일 형 변환은 서로 다른 목적의 변환을 같은 문법으로 표현
- C++ 형 변환 연산자는 변환 목적을 드러냄

연산자	목적
<code>static_cast</code>	관련 타입 사이의 일반 변환
<code>const_cast</code>	포인터나 참조의 상수성 추가·제거
<code>reinterpret_cast</code>	저수준 표현을 다른 타입으로 해석
<code>dynamic_cast</code>	다형 클래스 계층의 실행 시점 타입 확인

```
double speed{ 10.5 };  
int value{ static_cast<int>(speed) };
```

- 명시적 변환은 값 손실을 자동으로 막아 주지 않음



- 기존 열거형의 열거자는 선언된 범위로 들어옴
- 정수형으로 암시적 변환 가능
- 서로 다른 열거형에서 같은 열거자 이름 충돌 가능

```
enum class State
{
    Idle, Running, Finished,
};
State state{ State::Idle };
// int value{ State::Running }; // 오류
int value{ static_cast<int>(State::Running) };
```

구분	기존 열거형	enum class
열거자 범위	바깥 범위	열거형 내부
정수 변환	암시적 가능	명시적 변환 필요
이름 충돌	발생 가능	축소



● auto

- ◆ 초기값으로부터 변수 타입을 추론
- ◆ 타입을 없애는 기능이 아님
- ◆ 결정된 타입은 실행 중 변경되지 않음

```
auto count { 10 };           // 추론 타입: int
auto speed { 3.5f };         // 추론 타입: float
auto enabled{ true };        // 추론 타입: bool
auto nullptr{ nullptr };     // 추론 타입: std::nullptr_t
auto voidPtr{ (void*)nullptr }; // 추론 타입: void*
auto myPtr { (MyClass*)nullptr }; // 추론 타입: MyClass*

int*    intPtr1 = nullptr;    // 성공
float*  floatPtr = nullptr;   // 성공
MyClass* objPtr = myPtr;      // 성공: 동일한 포인터 타입 대입
// int*  intPtr2 = voidPtr;    // 오류: C++에서 void*는 암시적 변환 불가
int*    intPtr2
    = static_cast<int*>(voidPtr); // 성공: void*는 명시적 캐스팅 필수
```



- auto 포인터, 참조, const

- ◆ auto*
- ◆ auto&
- ◆ const auto&

```
int value[] { 10, 20, 40, 50 };
```

```
auto          decayedPtr = value;    // 추론: int*  
auto*         firstPtr  = value;    // 추론: int*  
auto&         arrayRef   = value;    // 추론: int (&)[4]  
const auto&   constRef   = value;    // 추론: const int (&)[4]
```

- decayedPtr / firstPtr : int*로 변환되어 배열 크기를 알 수 없음
- arrayRef : 원본 배열 참조, 배열 크기를 알 수 있음
- constRef : 원본 배열 참조, 배열 크기를 알 수 있고 원소 변경 불가



기능	목적
함수 오버로딩	같은 이름으로 매개변수 타입·개수가 다른 함수 구성
기본 인수	호출 시 일부 인수 생략
인라인 함수	헤더 정의와 함수형 매크로 대체
반환 타입 추론	return 문으로 반환 타입 추론





- 같은 이름으로 다른 기능
 - ◆ 매개변수 개수, 타입이 다를 때
 - ◆ 같은 역할의 함수를 일관된 이름으로 구성
 - ◆ 반환 타입만 다른 함수는 오버로딩할 수 없음
 - ◆ 호출 인수의 개수와 타입에 따라 함수 선택
 - ◆ 정확히 일치하는 타입, 정수 승격, 표준 변환 등이 고려됨
 - ◆ 여러 함수가 같은 정도로 적합하면 호출이 모호해질 수 있음

```
int Add(int left, int right)
{
    return left + right;
}
double Add(double left, double right)
{
    return left + right;
}
int Add(int first, int second, int third)
{
    return first + second + third;
}
```

```
int firstResult = Add(10, 20);
double secondResult = Add(1.5, 2.5);
int thirdResult = Add(10, 20, 30);
```



- 호출 의도를 분명하게 하려면
인수 타입을 명시적으로 맞추

```
void Process(long value);  
void Process(double value);  
  
// Process(10); // 모호함  
  
Process(static_cast<long>(10));
```





- 인수를 생략하면 기본값 사용
- 기본 인수는 오른쪽 매개변수부터 연속해서 지정

```
void PrintMessage(const char* message, int count = 1)
{
    for (int i = 0; i < count; ++i)
    {
        std::cout << message << '\n';
    }
}
PrintMessage("Hello");
PrintMessage("Hello", 3);
```

- 선언과 정의를 분리할 때 기본 인수는 함수 선언에만 작성
- 오버로딩과 함께 사용할 때 호출 모호성 주의

```
void Move(int x, int y = 0, int z = 0);

// void Move(int x = 0, int y); // 오류
```



- C의 함수형 매크로는 전처리 과정에서 문자 단위 치환
- 인수 식이 여러 번 평가되거나 연산자 우선순위 문제가 생길 수 있음
- C++에서는 같은 목적의 계산을 함수로 작성

```
inline int Square(int value)
{
    return value * value;
}
```

- 일반 함수이므로 타입 검사 수행
- 인수는 함수 호출 규칙에 따라 한 번 평가
- inline은 최적화 강제가 아님
- 헤더 파일에 동일한 함수 정의를 둘 수 있게 하는 언어 규칙으로 이해

```
inline int Add(int left, int right)
{
    return left + right;
}
```



- C++14부터 auto로 반환 타입 추론

```
auto Add(int left, int right)
{
    return left + right;
} // 반환타입:int
```

- 여러개인 반환문에서 모든 반환값 타입 일치

```
auto GetValue(bool condition)
{
    if (condition)
    {
        return 10;
    }
    else {
        // 오류: 다른 반환값과 타입 불 일치
        // return 20.0;
    }
    return 30;
}
```



- 후행 반환 타입은 반환 타입을 명시하거나 템플릿에서 복잡한 타입 관계를 표현할 때 사용

```
auto Multiply(double left, double right) -> double
{
    return left * right;
}
```





2.10 참조(Reference)

- Alias: 참조는 기존 객체에 붙인 다른 이름
- 선언할 때 참조할 대상을 지정해야 함
- 일반적인 좌측값 참조는 초기화 후 다른 객체를 참조하도록 변경할 수 없음
- 포인터처럼 *로 역참조하지 않음
- 호출할 때 주소 연산자 &를 작성하지 않음
- 상수 참조(const &)는 참조를 통해 원본 값을 변경할 수 없음 (읽기 전용)
- 일반 참조와 달리, 상수 참조는 리터럴(예: 10)이나 임시 객체도 참조할 수 있음
-

```
int value = 10;
// int& ref = 10;           // 오류: 비 const 참조는 임시 객체 참조 불가
const int& cref = 10;       // 가능: 임시 객체의 수명 연장
int& reference = value;
reference = 20;
std::cout << value << '\n';    // 20

int first = 10;
int second = 20;
int& reference = first;
reference = second;           // 참조 변경이 아니라 first에 값 대입
```



2.10.1 포인터와 참조의 기본 차이

- 참조도 내부 구현에서는 주소를 이용할 수 있음
- C++ 문법에서 참조는 객체의 별칭으로 다룸
- 널 상태가 필요하거나 선택적으로 대상을 전달해야 하면 포인터가 적합

구분	포인터	참조
선언 후 대상 변경	가능	불가능
널 상태 표현	nullptr 사용 가능	일반적으로 유효한 객체에 연결
대상 접근	*pointer	일반 변수와 같은 문법
호출 시 주소 전달	&value	value





2.10.2 값, 주소와 참조에 의한 전달

```
void IncreaseValue(int value)
{
    ++value;
}

void IncreasePointer(int* value)
{
    if (value != nullptr)
    {
        ++(*value);
    }
}

void IncreaseValue(int value)
{
    ++value;
}

void IncreasePointer(int* value)
{
    if (value != nullptr)
    {
        ++(*value);
    }
}

void IncreaseReference(int& value)
{
    ++value;
}
```

```
int first = 10;
int second = 10;
int third = 10;

IncreaseValue(first);
IncreasePointer(&second);
IncreaseReference(third);
```

- **first**는 10 유지
- **second, third**는 11로 변경



2.10.2 값, 주소와 참조 전달 비교

구분	값 전달 (Call by Value)	주소 전달 (Call by Address)	참조 전달 (Call by Reference)
함수 코드	<pre>void Swap(int a, int b) { int t = a; a = b; b = t; }</pre>	<pre>void Swap(int*a, int*b) { int t = *a; *a = *b; *b = t; }</pre>	<pre>void Swap(int&a, int&b) { int t = a; a = b; b = t; }</pre>
함수 호출	Swap(x, y);	Swap(&x, &y);	Swap(x, y);
전달되는 대상	인수의 값	객체의 주소	원본 객체에 대한 참조
원본 변경	변경되지 않음	변경됨	변경됨
매개변수 사용	일반 변수와 같음	*로 역참조	일반 변수와 같음





- 읽기만 필요한 객체는 상수 참조로 전달
- 객체를 복사하지 않음
- 참조를 통해 값을 변경하지 못하게 제한
- 문자열이나 컨테이너처럼 복사 비용이 큰 객체에 유용

```
#include <string>
#include <vector>

void Print(const std::string& text)
{
    std::cout << text << '\n';
}

void PrintItems(const std::vector<int>& items)
{
    for (int item : items)
    {
        std::cout << item << '\n';
    }
}
```

- **int, double**처럼 복사 비용이 작은 기본형은 값 전달이 단순
- 상수 객체와 비상수 객체를 모두 전달 가능



- 함수는 참조를 반환할 수 있음
- 반환된 참조는 원본 객체를 나타내므로 읽기와 변경 가능
- 참조 대상은 함수 호출 이후에도 살아 있어야 함

```
int& GetElement(int* values, int index)
{
    return values[index];
}
```

```
int values[3] = { 10, 20, 30 };
```

```
GetElement(values, 1) = 100;
```

```
// 이후 values[1]은 100
```

```
int& CreateValue()
```

```
{
```

```
    int value = 10;
```

```
    // 함수 종료 후 value의 수명이 끝나므로 댕글링 참조 발생
```

```
    return value;
```

```
}
```

- 지역 변수는 함수가 끝날 때 소멸
- 수명이 끝난 객체를 가리키는 댕글링 참조 발생



- 이름 공간은 이름 충돌을 줄이는 범위 관리 기능
- 출력 스트림은 타입에 맞는 출력 동작을 선택
- 서식 조정자는 적용 범위가 서로 다름
- 입력 스트림은 타입이 맞지 않으면 실패 상태 설정
- `clear()`는 상태 복구, `ignore()`는 버퍼 정리
- 기본형 크기와 정렬은 실행 환경에 따라 달라짐
- 문자형은 문자 인코딩의 코드 단위를 저장
- 부동소수점값은 근사 표현과 오차를 고려
- 초기화와 대입은 다른 과정
- `const`, `constexpr`, `constexpr`, `constexpr`의 목적 구분
- C++ 형 변환 연산자는 변환의 목적을 코드에 드러냄
- 참조는 기존 객체의 다른 이름이며 전달 방식 설계에 사용

